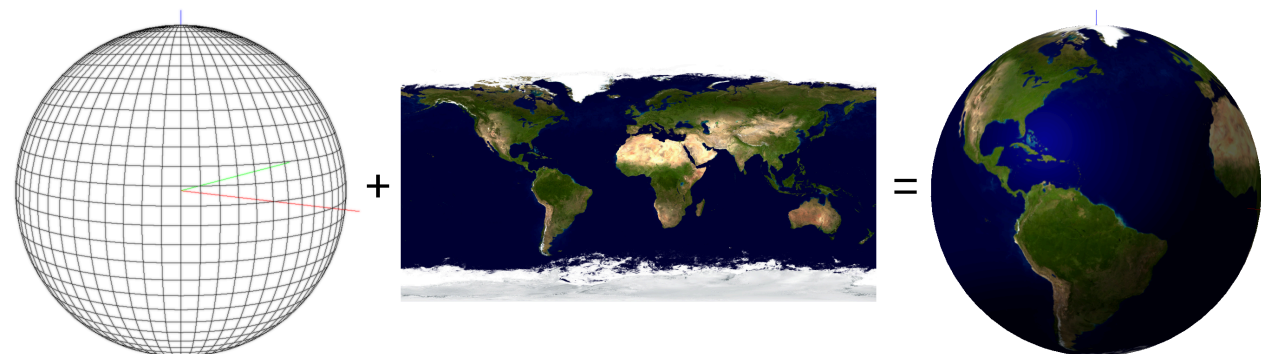


Computación Gráfica 2026

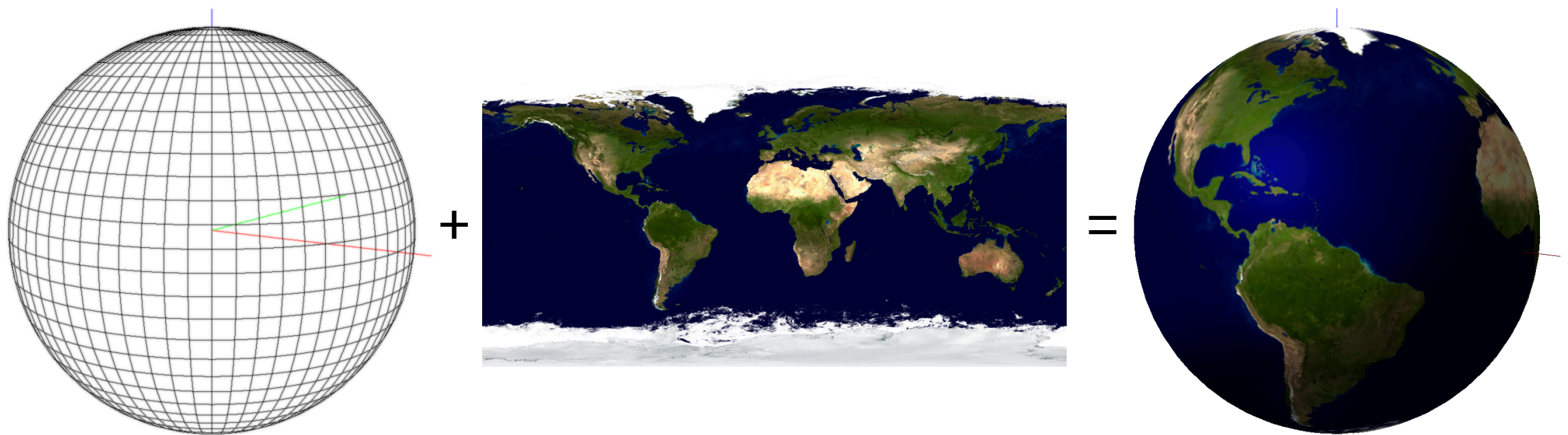
Unidad 6

Texturas



INTRODUCCIÓN

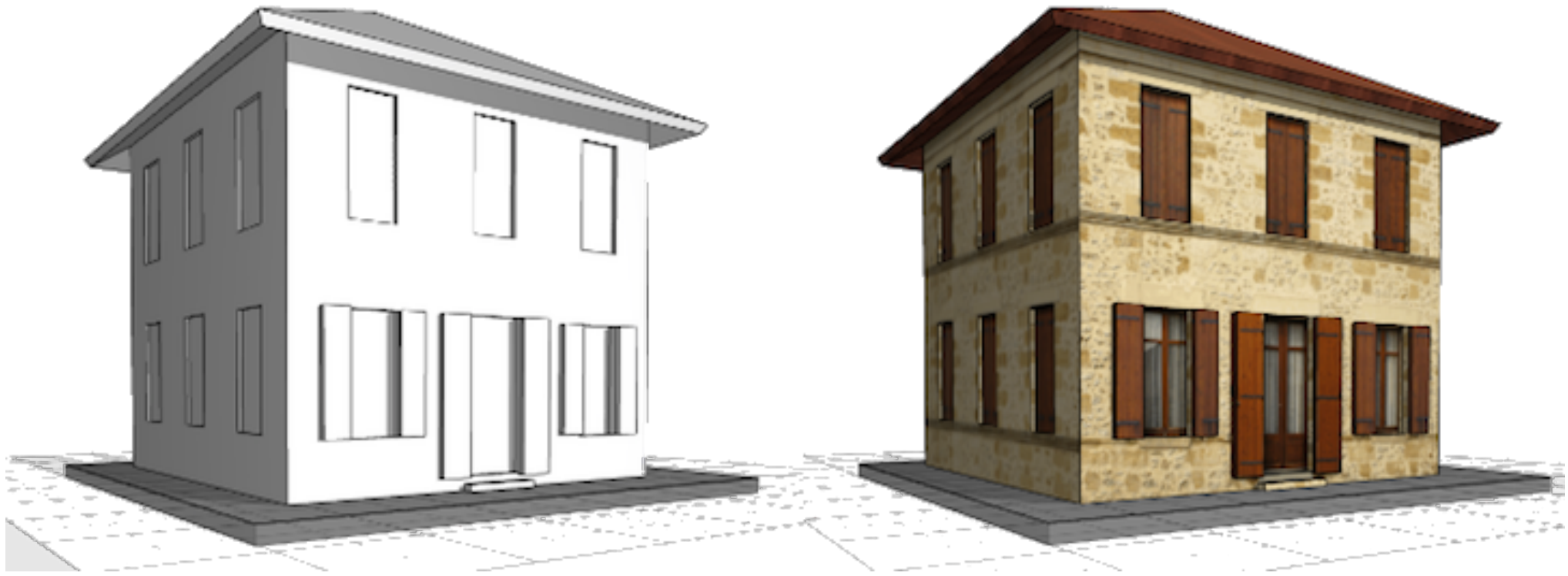
Una textura es una imagen que de alguna forma "mapeamos" sobre el modelo, y entonces podemos usar su "color" para algo en el fragment-shader.



Una textura es una imagen que se "pega" sobre un modelo para poder tomar de ella "valores" diferentes en cada fragmento. Estos valores podrían significar muchas cosas, pero el primer y más obvio caso de uso sería color: una textura es como una "foto" que se pega sobre una primitiva para que su color no sea el mismo en todo el triángulo, sino que simule algo más complejo y detallado.

INTRODUCCIÓN

Los modelos de iluminación por sí solos generan imágenes carentes de realismo.



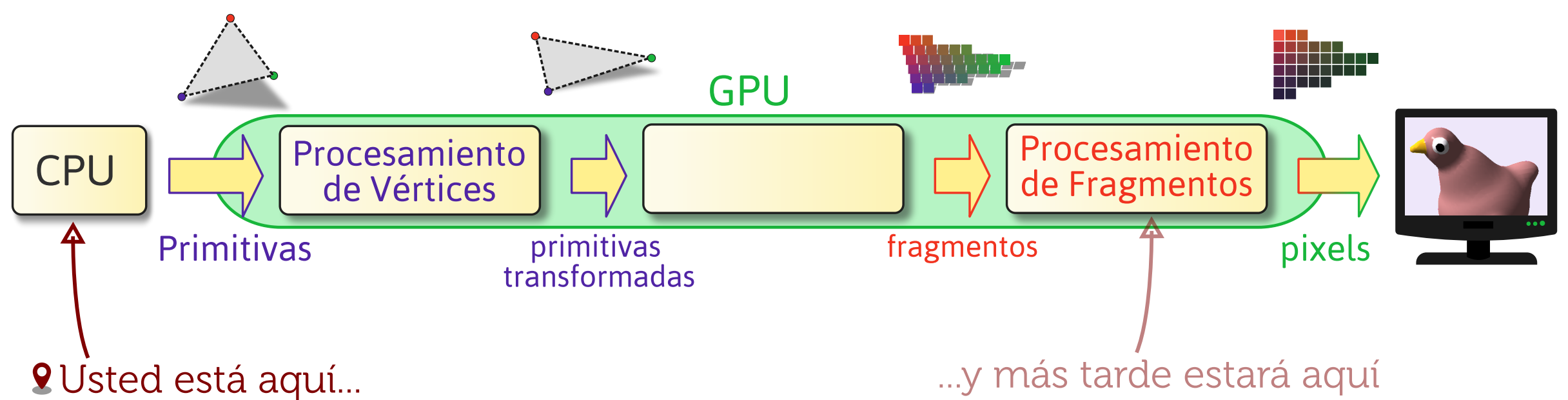
Capturar todos los detalles solo con "geometría" sería carísimo.

Un modelo de iluminación alcanza para dar una buena sensación de profundidad, pero no de alto realismo. Las superficies reales no son perfectamente lisas o parejas/constantes. Un modelo de iluminación sin texturas asume que en cada triángulo las propiedades del material son constantes (o varían linealmente como mucho).

Muchas superficies reales tienen detalles o imperfecciones propias del material. Por ejemplo, una pared no suele ser nunca realmente plana (sino que tiene cierto relieve) y rara vez su pintura es perfectamente pareja y sin manchas. En otros casos, la geometría es simplemente demasiado compleja (imaginen modelar los pastos de una cancha de fútbol) y necesitamos algún mecanismo auxiliar para simularla.

En ambas situaciones, sería impensable (o al menos generalmente muy costoso y poco práctico) capturar esos detalles con geometría (que la "forma" de la pared en verdad copie el relieve con triangulitos muy pequeños, o que el campo de fútbol esté modelado con millones de "pastos" individuales). Por ello se utilizan texturas, como una solución de compromiso muy conveniente, tanto para el renderizado como para el modelado.

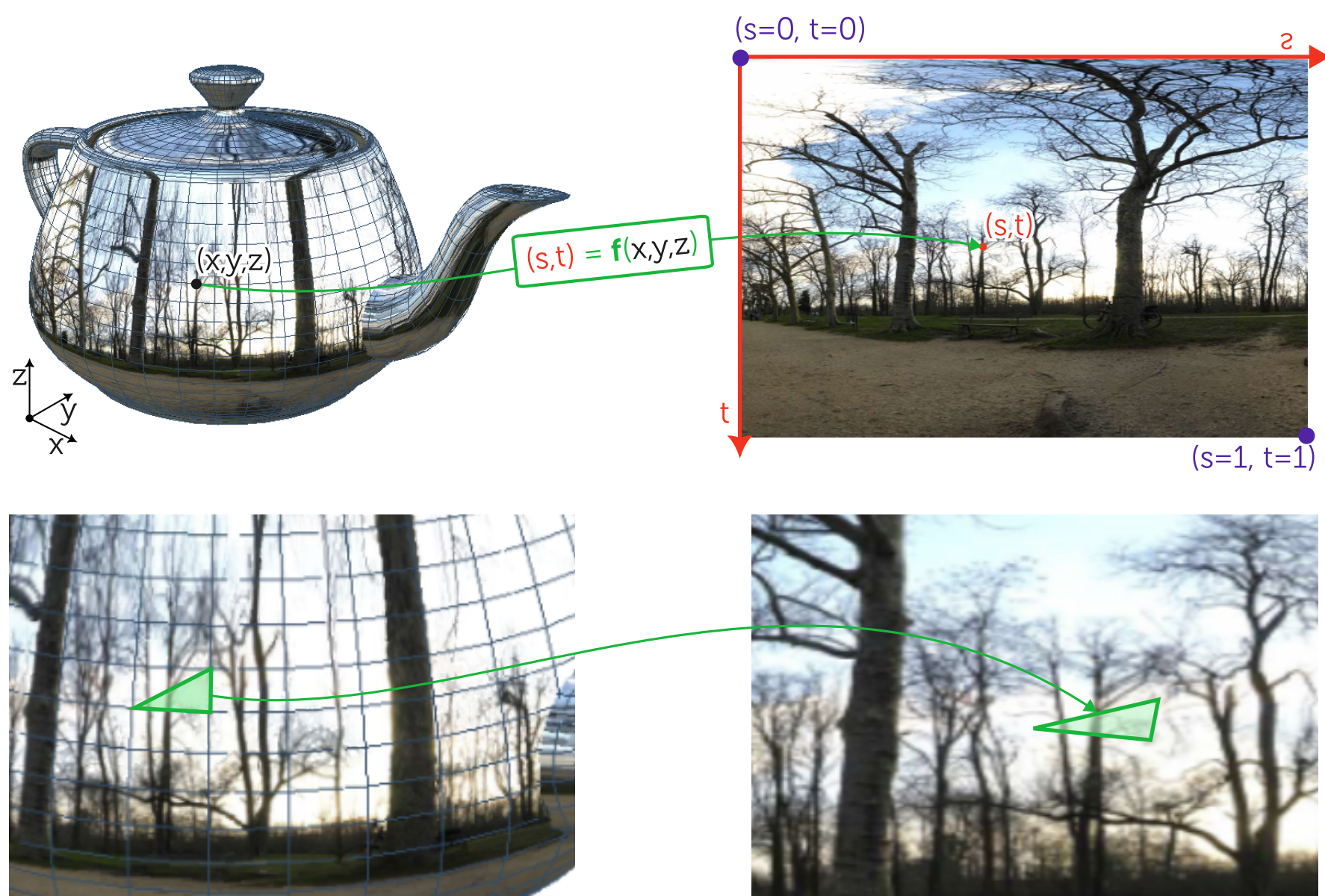
USTED ESTÁ AQUÍ



La aplicación debe definir qué texturas se van a usar y para qué, y especialmente cómo pegar la textura sobre cada modelo. Por esto, el "usted está aquí" comienza en la aplicación; y es de este "cómo pegar" (técnicamente el "mapeo") de lo que hablaremos la primera mitad de la clase.

Pero una vez definido ese mapeo, la verdadera aplicación de la textura se hace en el fragment shader. Se alimenta a la GPU con la imagen de la textura, las primitivas del modelo, y la información adicional para "mapear" lo primero sobre lo segundo (veremos que son las "coordenadas de textura" para cada vértice); y es el fragment shader el que efectivamente toma una "muestra" de la textura y usa sus colores para pintar el fragmento (por ejemplo, para alimentar la configuración del material en el modelo de iluminación). Por eso el "usted está aquí" señala especialmente la etapa del fragment shader. En la 2da mitad de la clase hablaremos de los problemas y detalles en esta etapa.

MAPEO DE TEXTURAS



Notar que para pegar la imagen sobre la tetera, el mapeo que necesito no es el que lleva la imagen a la tetera, sino exactamente el inverso: el que lleva los triángulos de la tetera a la textura. Porque la pregunta será luego en el fragment-shader ¿de qué color pinto este fragmento?, lo cual depende de ¿a qué punto de la textura tengo que ir a buscar el color? Generalmente esa respuesta por fragmento se interpola a partir de respuestas previas por vértice; entonces la pregunta usual es ¿a qué punto de la textura se corresponde cada vértice? Por esto decimos que al utilizarla necesitamos el mapeo de los triángulos (sus vértices) a la imagen y no al revés.

Por esto generalmente la información de entrada para la GPU incluirá el mapeo por vértice: junto con las coordenadas xyz de cada vértice, además de normal, material, etc, también habrá asociadas otras coordenadas (llamadas "de textura", generalmente asociadas con las letras s y t). Entonces es responsabilidad de la aplicación obtener esas coordenadas para pasarle a la GPU.

Cuando se renderizan modelos complejos, esa información suele estar en el archivo del modelo (por ejemplo, en el obj en la mayoría de los TPs), pero alguien (alguna aplicación) la tiene que haber generado en primer lugar. Estudiaremos algunas formas de generar esta información (de "mapear" la textura).

Por convención se utilizan las letras s y t para ubicar un punto en la imagen, y se utilizan valores normalizados para independizarse de la resolución de la misma. Entonces (0,0) será el punto de arriba a la izquierda, y (1,1) el de abajo a la derecha (en el mundo raster, por razones históricas, el eje vertical crece para abajo, y no para arriba como estamos acostumbrados en matemáticas).

Resumiendo, necesito las funciones $s=f(x,y,z)$ y $t=q(x,y,z)$: si en verdad f y g son funciones matemáticas puede verse como un mapeo automático; mientras que si sale de un "tabla" sin una regla general se dirá que es manual.

MAPEO MANUAL VS. AUTOMÁTICO

• Manual:

- en el modelo, cada vértice tiene como dato sus coordenadas s,t
- se interpola el s,t para los fragmentos 🚨 *interpolación hiperbólica*
- implica guardar mucha información
- ¿de dónde salen? ¿cómo se generaron al crear el modelo?

• Automático:

- las coordenadas s,t para cada "punto" (vértice o fragmento) se calculan mediante una función
- solo hay que guardar unos pocos parámetros de la función
- en general solo aplica a modelos (en algún sentido) "simples"

Se dice que el mapeo es "manual" cuando el s/t de cada vértice es dato, impuesto uno por uno. Mientras que se dice "automático" cuando se calcula mediante alguna fórmula: usualmente se diseña una función que dadas las coordenadas del vértice (x,y,z) genera las coordenadas de textura (s,t).

Para la GPU el mapeo casi siempre parece manual, ya que le llega un par st diferente para cada vértice (aunque hayan sido previamente calculados con una fórmula). Esos valores se interpolan para cada fragmento (con interpolación hiperbólica, lo que se dice "perspective-correct") y luego el fragment-shader los usa.

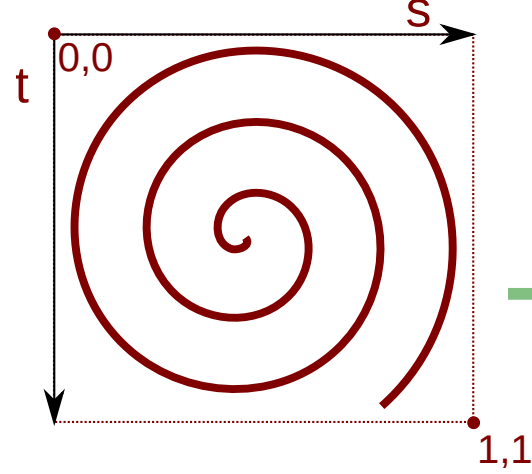
Sin embargo, desde la aplicación, esos datos se pueden haber generado efectivamente de forma manual (si son pocos, el programador los podría definir o calcular individualmente); o de forma automática (si el programador aplicó una función o algún algoritmo que genera los valores automáticamente para todos los vértices).

También, en algunos casos de mapeos no tan complejos, se podría implementar la fórmula que los calcula automáticamente directamente en el shader, para no necesitar recibir tantos datos desde la aplicación (solo algunos parámetros de la función, pero ya no un par st por vértice).

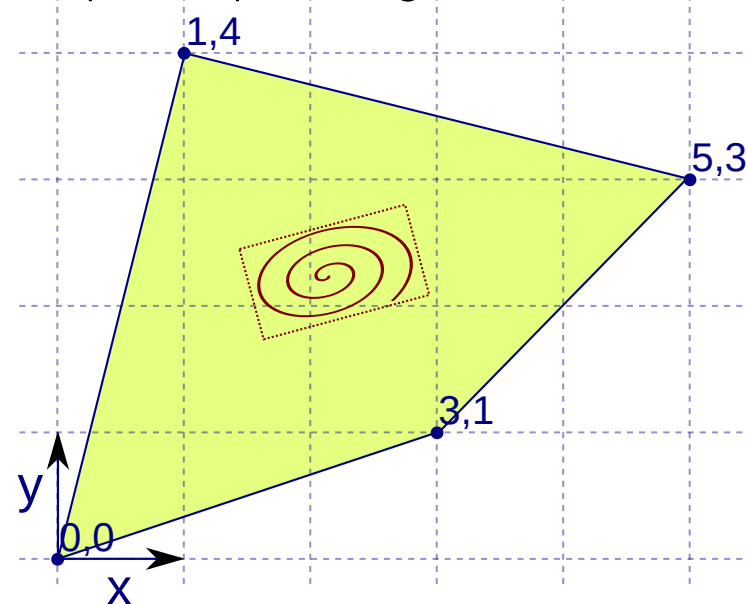
Nosotros diremos que es un mapeo automático cuando lo generamos con alguna fórmula o algoritmo para todos los vértices, sin importar si se implementa en CPU o GPU. Veremos en breve con cierto detalle algunos mecanismos de mapeo automático para casos relativamente simples.

MAPEO MANUAL

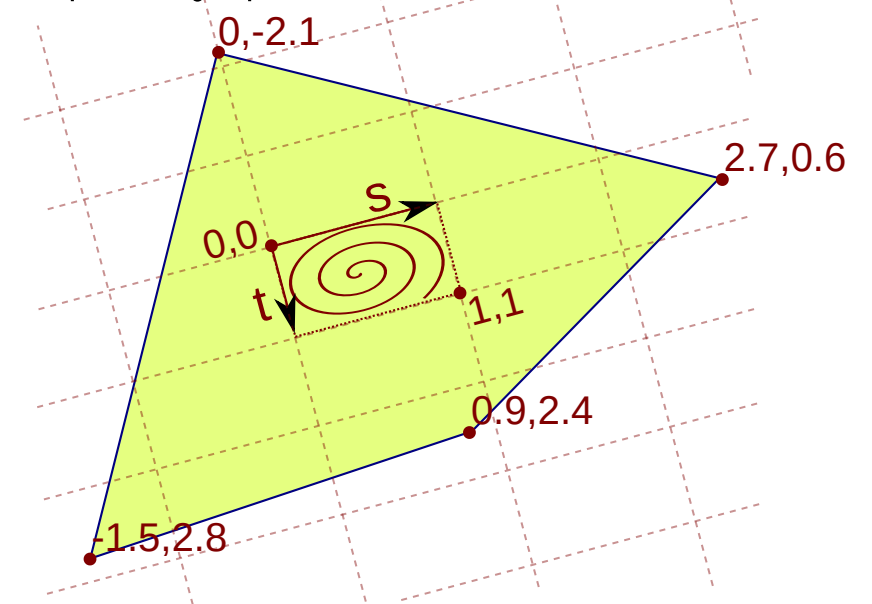
Textura:



Lo que se quiere lograr:



Lo que hay que "medir":



```
vector<glm::vec2> pos = { {0, 0}, {3, 1}, {5, 3}, {1, 4} }; // x,y  
vector<glm::vec2> tc = { {-1.5, 2.8}, {0.9, 2.4}, {2.7, 0.6}, {0.0, -2.1} }; // s,t
```

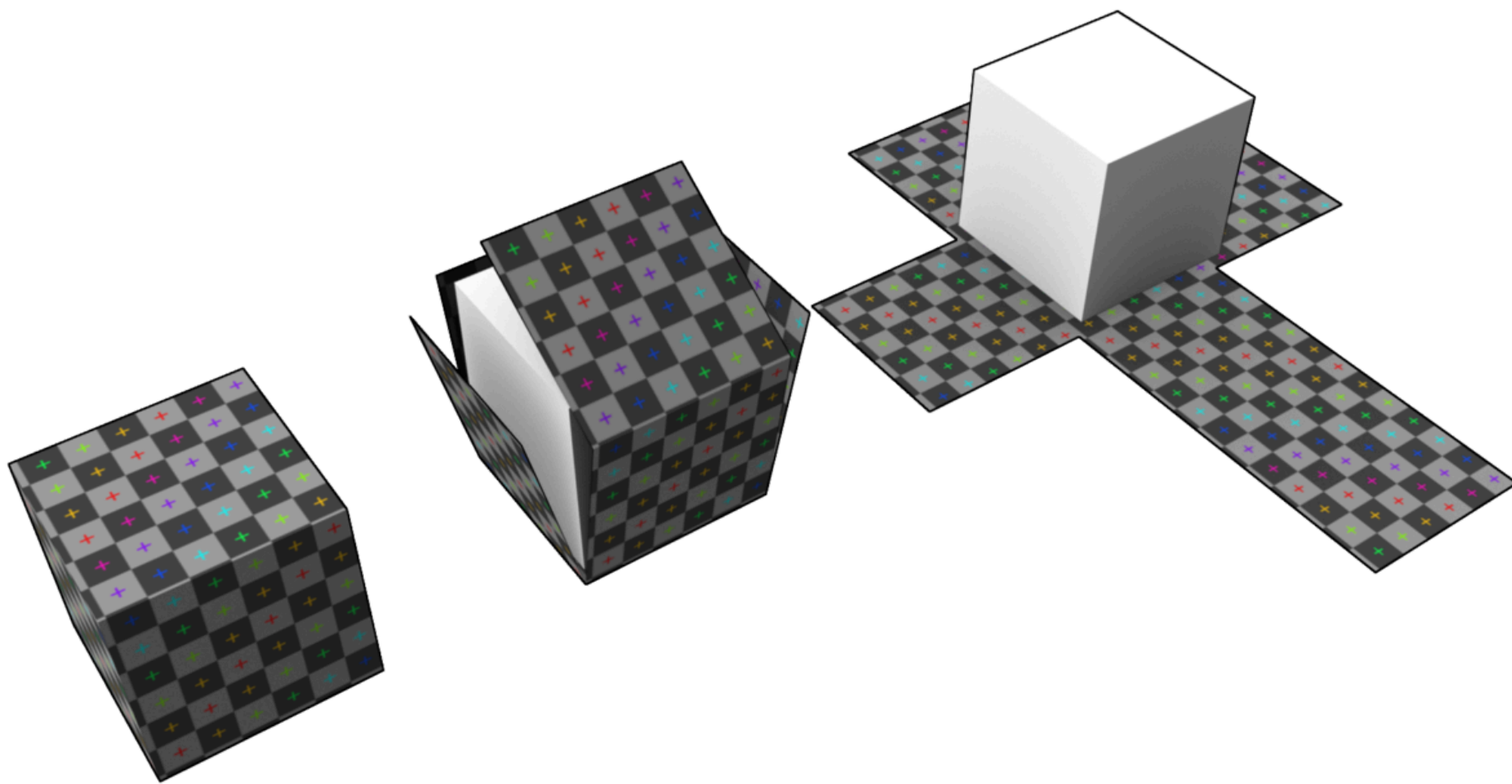
v_0 v_1 v_2 v_3

⚠ Se deben definir las coordenadas de textura **en los vértices**

Hacer un mapeo manualmente es indicar individualmente las coordenadas para cada vértice. Solo tiene sentido práctico si tengo un objeto con muy pocos vértices. Suponiendo que sé dónde y cómo quiero que quede la imagen en relación al objeto, lo que hay que hacer es: en base a la imagen encontrar la base del sistema st, y "medir" los vértices de la primitiva según esa base.

Nuevamente: la pregunta final no es en qué coordenadas (xyz) quiero la imagen (lo que usualmente es el dato), sino qué coordenadas de textura (st) tendrán los vértices (lo contrario). La información ingresa al pipeline "por vértice".

MAPEO UV

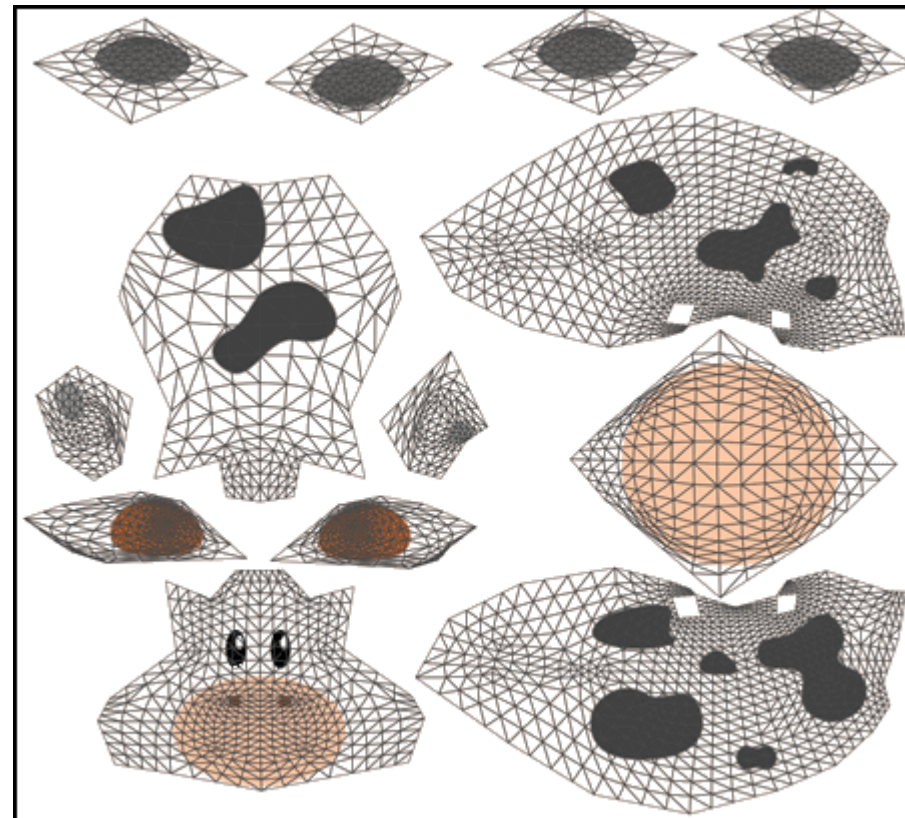
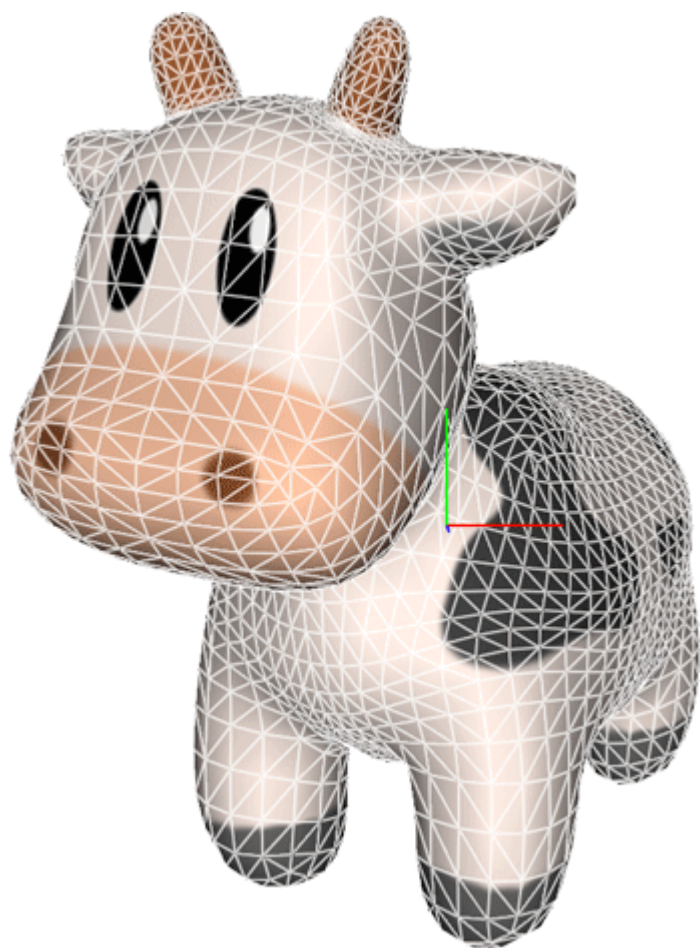


La idea aquí es "abrir" (recortando en algunos lugares) a la malla y aplastar las partes sobre un plano para definir las coordenadas de textura.

En general en el contexto de algoritmos de renderizado (los programadores) nos referimos a las coordenadas de textura (en el caso 2D) como s y t , estando el $0,0$ en la esquina superior derecha. En los software de diseño (para el usuario) se suele llamar a las coordenadas u y v (y a veces con el $0,0$ abajo a la izquierda), y de ahí el nombre.

Aquí hay una caso de los más simples, donde todas las caras del cubo terminan organizadas en una especie de T plana. El cubo tiene aristas bien definidas por donde se puede "cortar" la superficie para "abirla" y cada cara es perfectamente plana por lo cual se proyecta sin problemas (sin deformaciones) sobre la imagen.

MAPEO UV

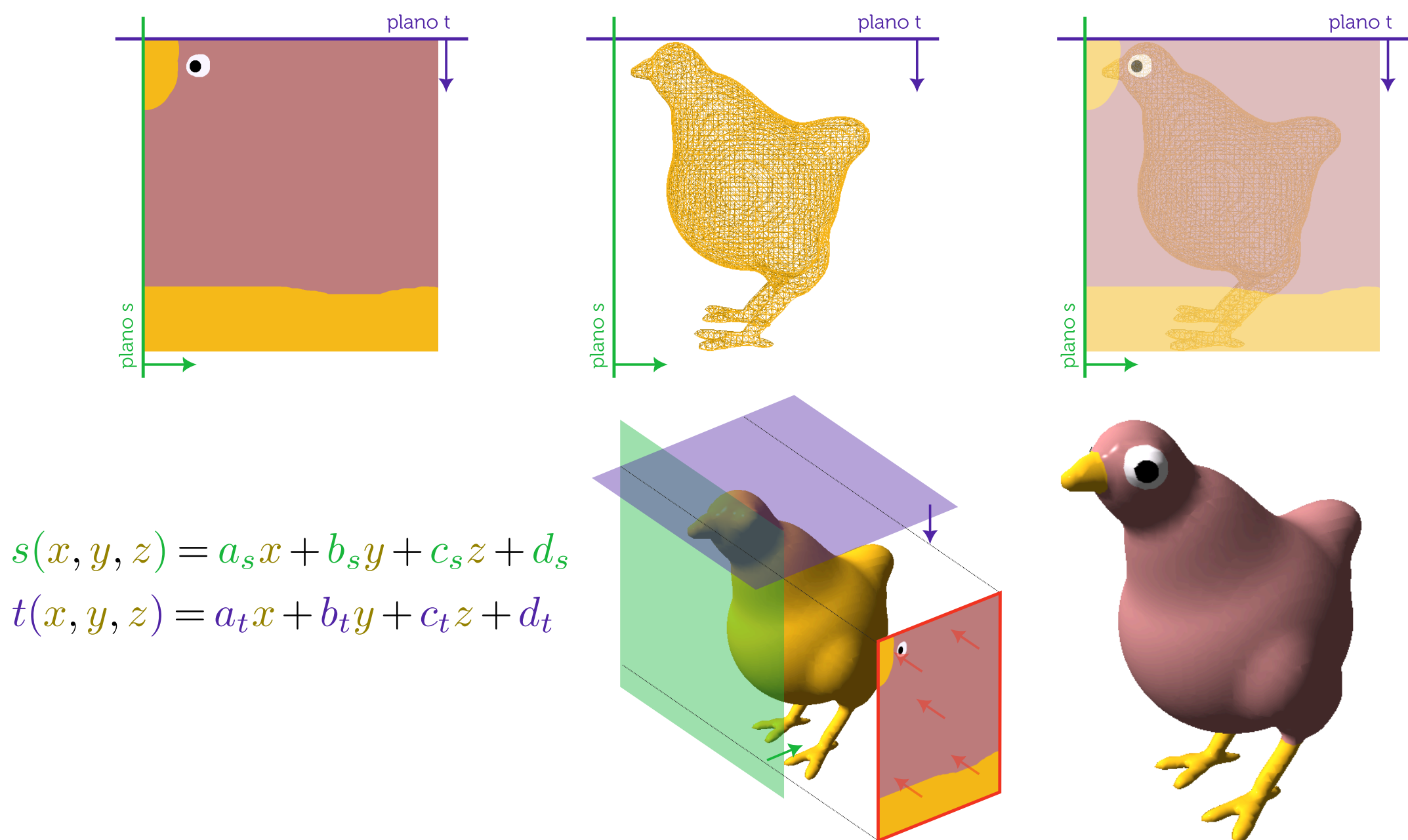


En un caso más real, no es simple decidir cómo recortar la superficie y cómo organizar las partes en la imagen.

Del recorte dependerá la "deformación" al aplastar cada parte: si son secciones más o menos planas, al llevarlas a la imagen casi no se deforman; o, lo que es lo mismo, el dibujo de la imagen casi no se deforma al "pegarlo" sobre el objeto.

De cómo se distribuyan las partes en la imagen dependerá que aprovechemos al máximo la memoria/resolución disponible, ya que habrá píxeles de esa imagen donde no se proyecte ninguna parte del objeto y entonces se desperdicien; y además las partes que ocupen mayor superficie en la imagen dispondrán de mayor resolución.

MAPEO AUTOMÁTICO: PLANO



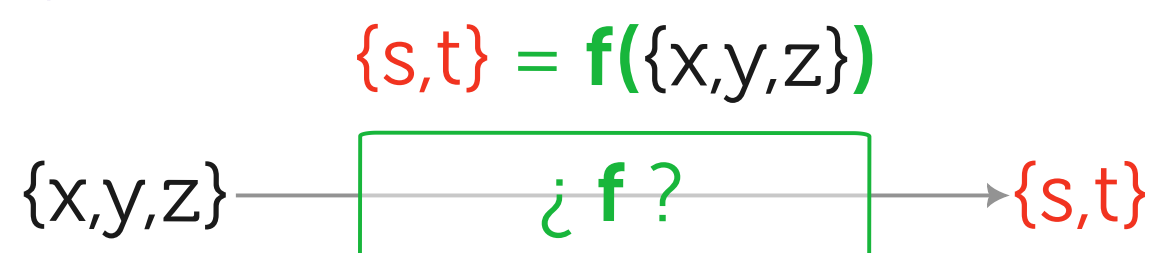
El mapeo plano obtiene un efecto similar al de proyectar la textura sobre el objeto (sería exactamente así si el cañón estuviera en el infinito). Para entender el efecto, se puede imaginar que se ubica la imagen en cierto lugar del espacio, y luego los téxeles se proyectan hacia el objeto siempre en dirección perpendicular a la imagen.

Sin embargo, cuando se implementa, se suele pensar de otra forma: definimos las funciones para calcular s y t como funciones que miden la distancia a dos planos (uno para s y otro para t) y usan esas distancias como coordenadas. Lo confuso aquí es que ninguno de los planos que hay que definir para estas funciones es el plano que contiene la imagen a proyectar, sino que son dos planos perpendiculares al mismo.

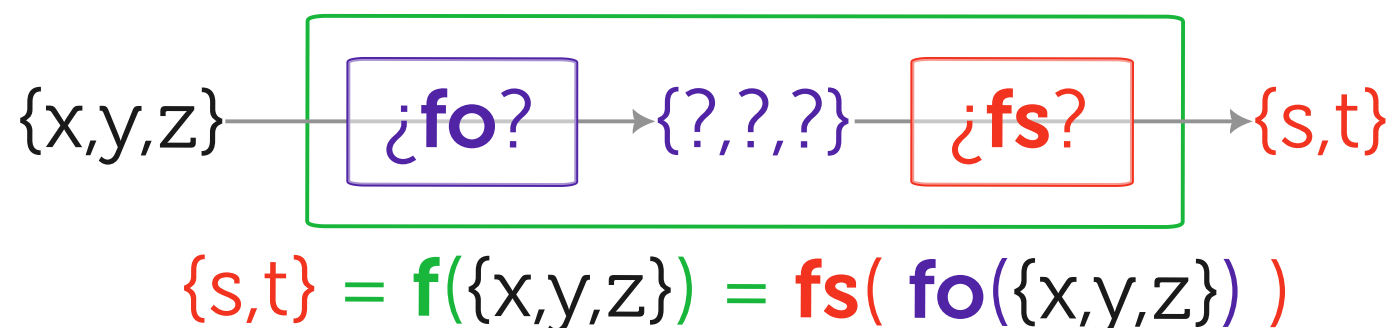
La ecuación del plano en 3D $ax+by+cz+d=0$ nos plantea una igualdad que se debe cumplir para todo punto xyz que pertenezca efectivamente al plano. Para interpretarla geométricamente, los coeficientes a , b y c definen la normal del plano, y entre los infinitos planos (paralelos entre sí) que podrían tener esa normal, el coeficiente d determina cual. Si colocamos en la expresión a la izquierda del igual un punto xyz que no pertenezca al plano, el valor calculado no será 0; sino que tendrá un signo dependiendo de en qué lado del plano está el punto, y el valor absoluto crecerá a medida que el punto se aleja del plano. Ese valor resultado es el que se usa como coordenada de textura. Donde sea 0 comienza la textura, y medida que el punto se aleja la coordenada crece (o decrece, depende para qué lado). La velocidad a la que crece es ajustable, ya que el cálculo no da exactamente la distancia (a menos que a,b,c sea un vector unitario), pero es proporcional, y controlando esa proporcionalidad (escalando esa normal) controlamos el "tamaño" de la textura.

MAPEO AUTOMÁTICO: EN DOS PASOS (O-MAPPING)

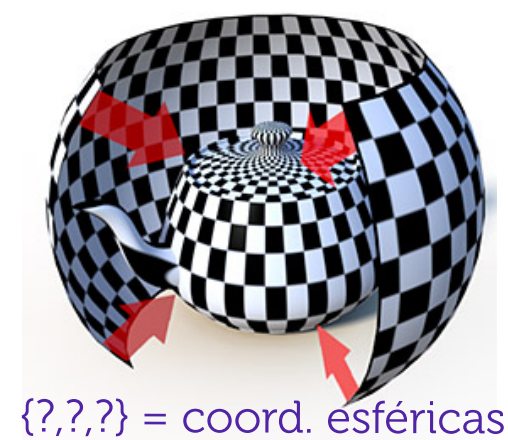
En un paso:



En dos pasos:



Ejemplos (dos pasos):



Algunas formas un poco más avanzadas de mapeo automático utiliza un paso intermedio: en lugar de pensar cómo llevo el vértice directamente a la imagen, lo llevo primero a la superficie de otro objeto más simple, y luego a la imagen. La idea es que en ese otro objeto más simple (que no es real, no hay malla ni primitivas, es solo conceptual) resulta más fácil encontrar una función para s y t .

Los dos casos más habituales son el mapeo cilíndrico y el mapeo esférico. Así como en coordenadas cartesianas un rectángulo alineado con un eje sería muy simple y natural de mapear, si expresamos el punto en coordenadas cilíndricas/esféricas será natural mapear un cilindro/una esfera a s y t .

Supongamos en coordenadas cartesianas, que hay que mapear la imagen sobre un cuadrilátero de lados alineados con los ejes x a y , sobre $z=0$: s será una simple función lineal (o afín) de x ; t lo será de y . De igual forma, en coordenadas cilíndricas podemos obtener s como una función lineal del ángulo y t de la altura; o en coordenadas esféricas una de cada ángulo.

Entonces, para calcular las coordenadas de textura de un vértice, primero lo llevamos al objeto intermedio, y luego a s y t . Hay muchas formas de pasar desde el vértice original al punto sobre la superficie imaginaria intermedia. En el caso de mapeo esférico y cilíndrico, lo más simple es proyectar el vértice según la dirección del radio, entonces las coordenadas que importan en el nuevo sistema son las mismas para el vértice original que para el punto imaginario proyectado (porque la diferencia estaría solo en el radio, y es la única coordenada que no participa en las funciones de s y t). Otras formas de pasar del vértice original a la superficie intermedia sería por ejemplo proyectarlo según la dirección de su propia normal, pero eso implicaría algo más de trabajo (el cálculo de una intersección).

ENVIROMENT MAPPING

Usar texturas para simular el reflejo del ambiente sobre un objeto:



Otra forma más complicada pero muy útil de pasar del objeto a la superficie intermedia es según la dirección en que "rebota" el rayo de la vista (el rayo reflejado). Con esta forma, la textura no queda "pegada" (fija) al objeto, ya que al mover la vista/cámara cambia ese rayo reflejado y entonces cambian las coordenadas de textura para un mismo punto. Esto genera el efecto de un espejo, y se utiliza para simular reflejos (algo que un modelo local como Phong nunca podría capturar). La textura debe contener para ello una "foto del ambiente" que hay alrededor del objeto y que se debe ver reflejado sobre el mismo.

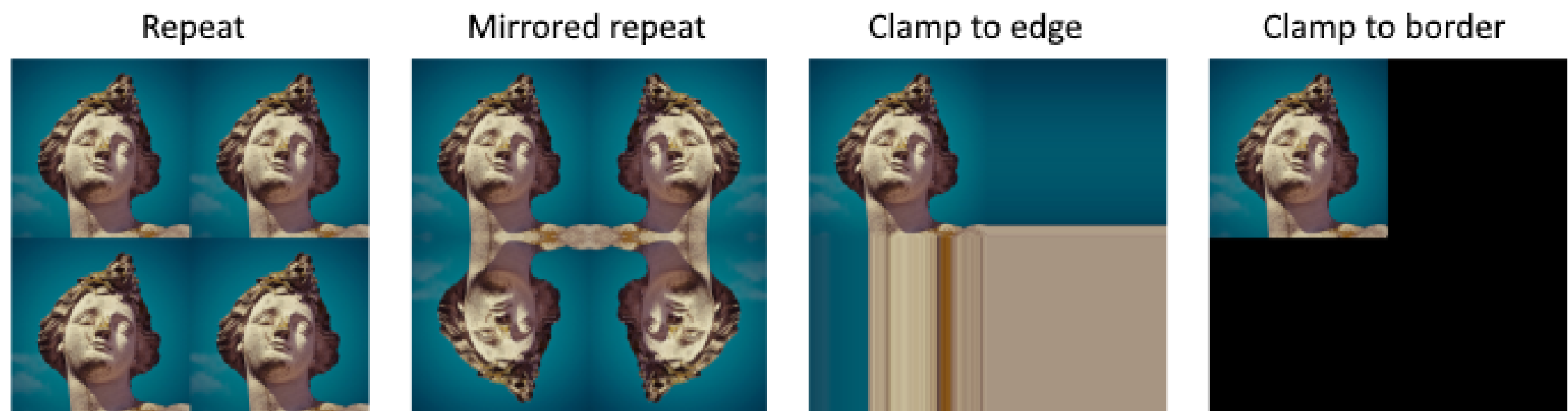
Hay distintas técnicas para lograr este efecto que se conoce como "Environment Mapping" (o mapeo "ambiental"). Lo más habitual es hacer algo similar al mapeo en 2 partes, envolviendo al objeto con otro objeto imaginario más simple sobre el cual se proyecta la textura del ambiente (usualmente un cubo, con una textura como la de abajo a la derecha, lo que se conoce como "cube map").

Cuando, por ejemplo, en un juego de carreras vemos las nubes o los árboles de alrededor de la pista reflejados sobre la chapa del auto, suele ser esta la técnica con que se logra. Puede ser que se use una textura genérica de cielo y entonces el reflejo no sea realmente lo que vemos en la pista (juegos muy viejos), o que el juego internamente renderice las vistas que necesita para armar el cubo y pueda así ir actualizando la textura. ¿Esto último implicaría 7 renderizados por frame? ¿el que vemos y 6 previos para armar primero el cubo? En realidad cuando se hace esto, la textura se actualiza con menos frecuencia (el trabajo de actualizarla se distribuye a lo largo de vario frames) y probablemente con menor resolución y menor nivel de detalle en los objetos

Nota: En los juegos más recientes, puede ocurrir que si se tiene una GPU lo suficientemente moderna se utilice RayTracing en lugar de una textura para estos efectos, pero no es el caso que estudiaremos en esta unidad.

WRAPPING/TILING

¿Qué hacer cuando s/t están fuera del rango [0;1]?



Finalmente, resta decir cómo funcionará el mapeo cuando el cálculo (o la asignación) de las coordenadas s y t de valores por fuera del rango $[0;1]$. Las opciones más comunes son:

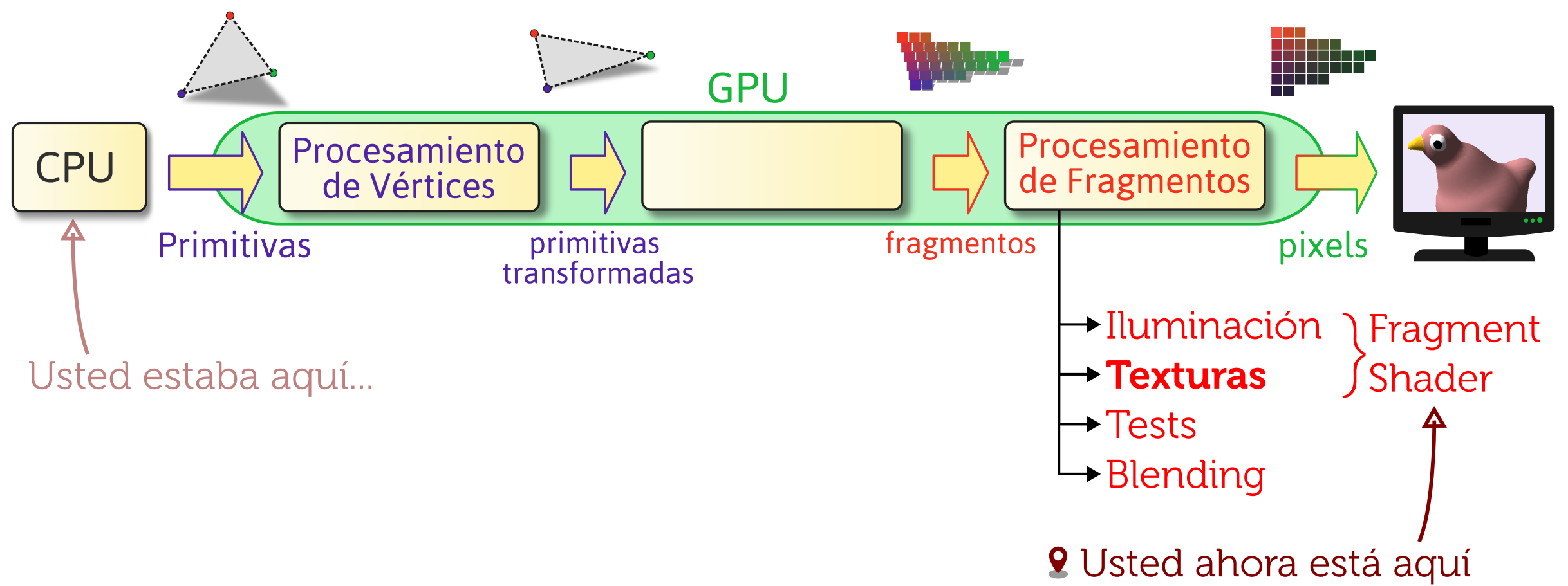
- repetir la textura periódicamente. Por ejemplo, entre $s=1$ y $s=2$ volvería a pasar lo mismo que entre $s=0$ y $s=1$, entonces $s=1.4$ sería lo mismo que $s=0.4$. El método tomará la parte fraccionaria de la coordenada de textura. Si en un juego de fútbol, por ejemplo, queremos poner el pasto sobre la cancha, en lugar de utilizar una textura enorme con hectáreas de pasto, podemos tener una más chica que cubra solo unos metros y repetirla con esta técnica. Lo mismo puede ocurrir con las vetas en una madera, los ladrillos en una pared, etc.

- repetir espejando: Al repetir, por ejemplo en s , si la última columna de texeles no es "compatible" con la primera, se notará visualmente el "corte" donde termina una "copia" de la textura y empieza la otra. En cambio, si las repeticiones impares se espejan, siempre la primera columna de una copia coincidirá con la última de la siguiente, disimulando la unión.

- clamp: En este caso, todo valor menor a 0 se considera 0 y todo valor mayor a 1 se considera 1. El efecto es que por fuera de $0...1$ se repetirá la primera o la última fila/columna de texeles. Si se quiere por ejemplo, usar una textura para pegar un calcomanía o algo similar sobre una superficie (imaginen que una manzana sobre la tapa de una notebook), se puede poner en la primera y última fila y columna $\alpha=0$ (100% transparente) y usar clamp, para que la manzana se vea una sola vez y en el resto de la tapa la textura no tenga efecto.

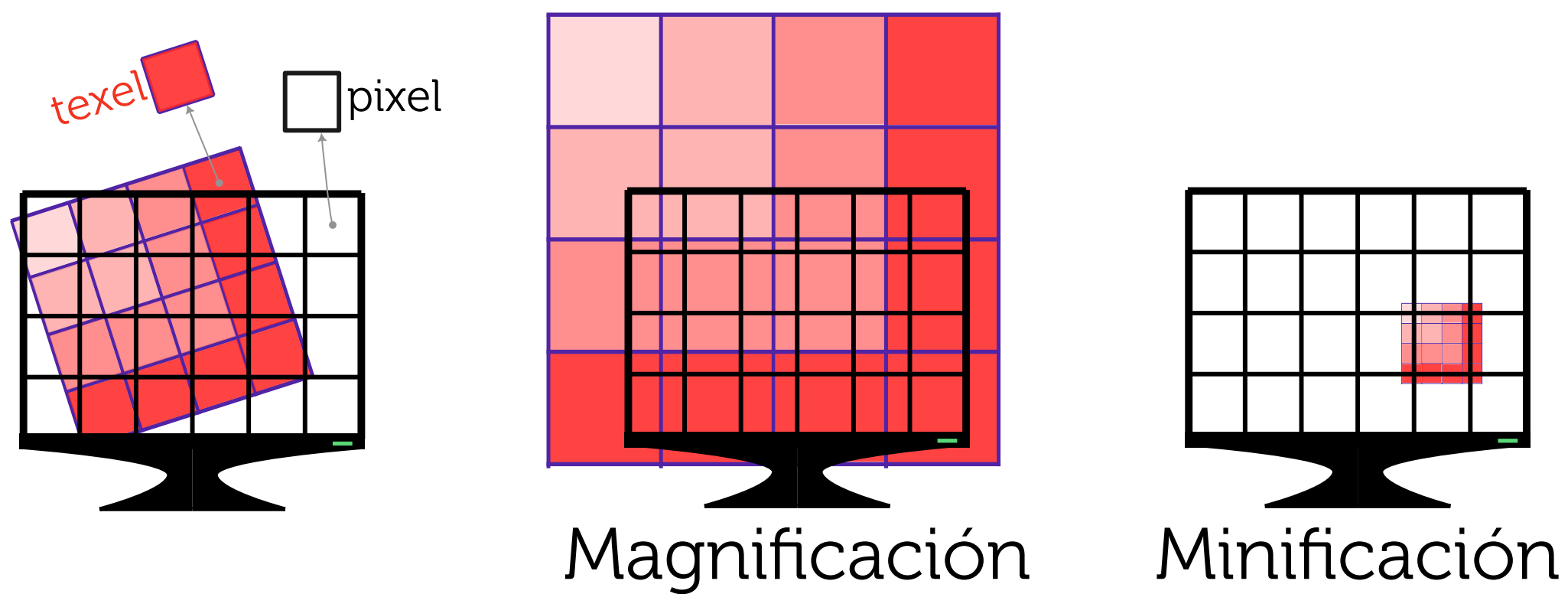
- clamp to border: cuando se dice "clamp" usualmente se refiere al método anterior ("to edge"), pero también se puede simplemente definir un color adicional además de la textura, para usar "por defecto" cuando nos salimos de la misma.

USTED ESTÁ AQUÍ



Resuelto el mapeo, el fragment shader recibirá las coordenadas de textura (interpoladas en la rasterización a partir de las que traían los vértices) o podrá calcularlas (podría implementarse directamente allí una fórmula de mapeo automático). Ahora analizaremos qué ocurre en esta etapa al utilizarlas.

EL PROBLEMA DEL FILTRADO



⚠ *Los píxeles no suelen coincidir en tamaño/alineación con los téxeles.*

El primer problema será que en general los téxeles de la textura no tendrán una correspondencia perfecta con los píxeles de la imagen: una "grilla" respecto de la otra puede estar escalada, rotada, o transformada/distorsionada de otra forma. Esto hará que las coordenadas de textura de cada fragmento no señalen "exactamente" un texel. Recordemos que las coordenadas son reales entre 0 y 1, mientras que en la textura la fila/columna de cada texel es un valor discreto.

Al momento de "muestrear" la textura, en el fragment shader tendremos una función que recibe un "sampler" y las coordenadas, y nos retorna el color: `vec4 tex = texture(colorTexture, fragTexCoords);`. El "sampler" (que es una entrada, un `uniform` en GLSL) representa a una textura en particular y tiene cierta configuración sobre cómo obtener el color para un dado `st`.

Las coordenadas de textura del fragmento (reales `s,t`), llevadas a fila/columna de la imagen de la textura (a enteros, lo que implica algún tipo de redondeo) indicarán un determinado texel. El sampler usualmente retornará un valor interpolado a partir de varios téxeles cercanos, y no directamente el color de un único texel original. Esto se hace para disimular algunos problemas que veremos a continuación, y se denomina "filtrado".

AMPLIACIÓN O MAGNIFICACIÓN: NEAREST VS. LINEAR



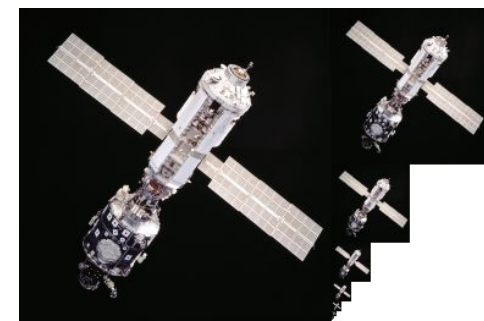
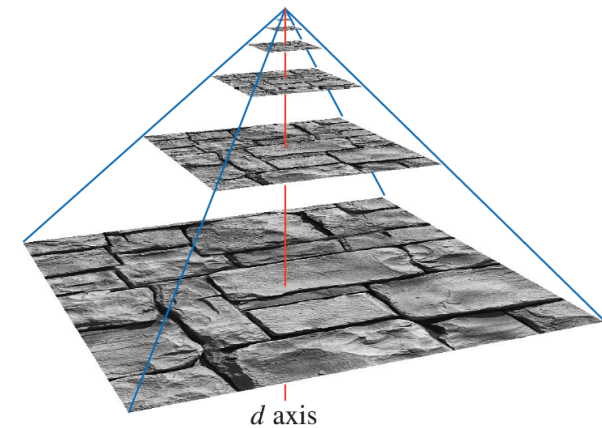
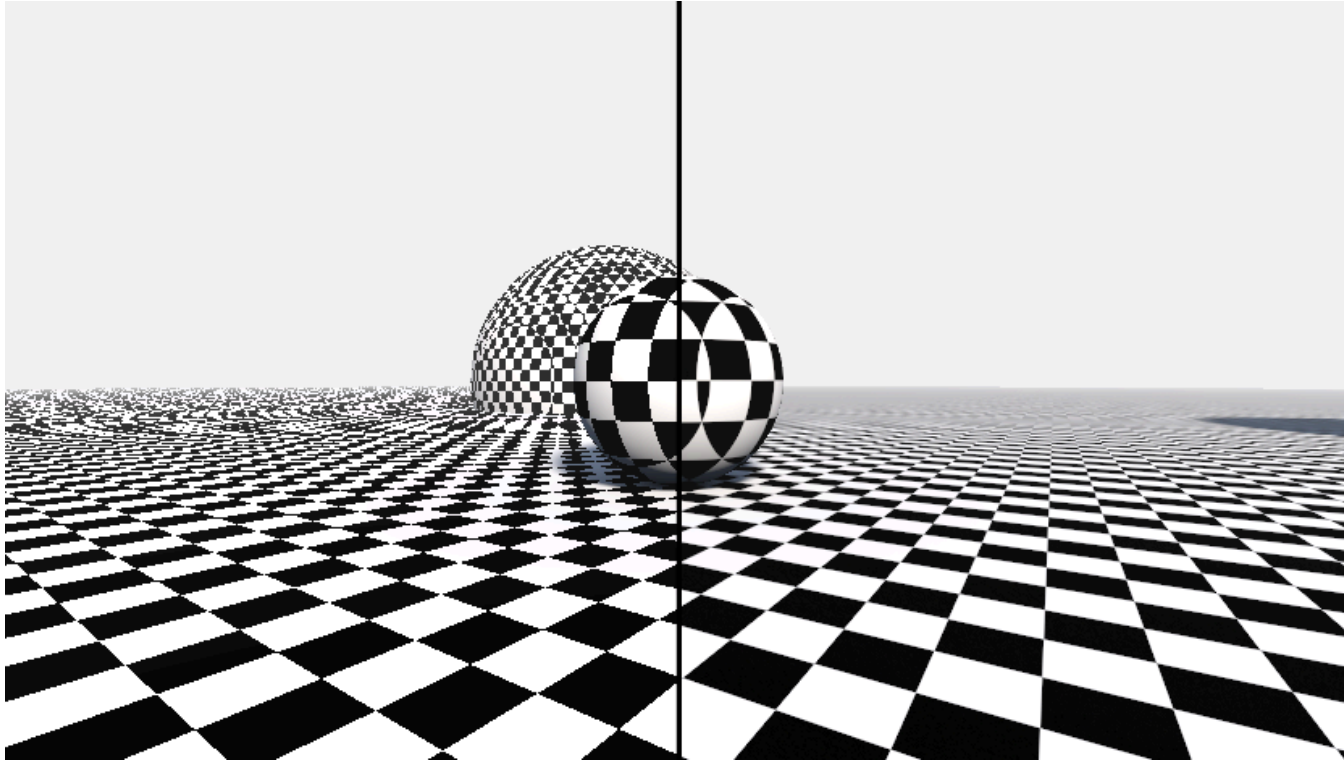
Cuando la textura se proyecte "muy grande" en pantalla (tamaño de los téxeles mayor al de los píxeles), si para cada fragmento simplemente tomamos el color de un téxel, se verá "pixelada". Grupos de fragmentos cercanos tomarán su color de un mismo téxel y por lo tanto tendrán el mismo color, y así el salto de téxel a otro será más evidente.

Supongamos por ejemplo una textura con 4 columnas de téxeles: todos los fragmentos con coordenada s entre 0 y $1/4$ tomarían su color de la primera columna, con s entre $1/4$ y $2/4$ de la segunda, entre $2/4$ y $3/4$ de la tercera, y entre $3/4$ y 1 de la cuarta. Esto haría el filtro que en OpenGL se conoce como "Nearest" (toma el téxel "más cercano").

Sin embargo es razonable pensar que en ese ejemplo un fragmento con $s=1/4$ debería tomar un promedio entre los colores de las dos primeras columnas (esa coordenada de textura cae justo en medio de ambas). Cuando esto se resuelve mediante interpolación lineal entre dos fragmentos, se lo llama "Linear". Como usualmente esta configuración (elegir usar linear o nearest) se hace de igual forma para ambas coordenadas (para s y para t), el filtro Linear termina siendo en realidad equivalente a una interpolación bilineal.

REDUCCIÓN O MINIFICACIÓN: MIPMAPS

Versiones reducidas de la textura original que se precaculan para interpolar "más" téxeles.



✓ *ahora podemos elegir un nivel donde téxel y pixel sean parecidos*

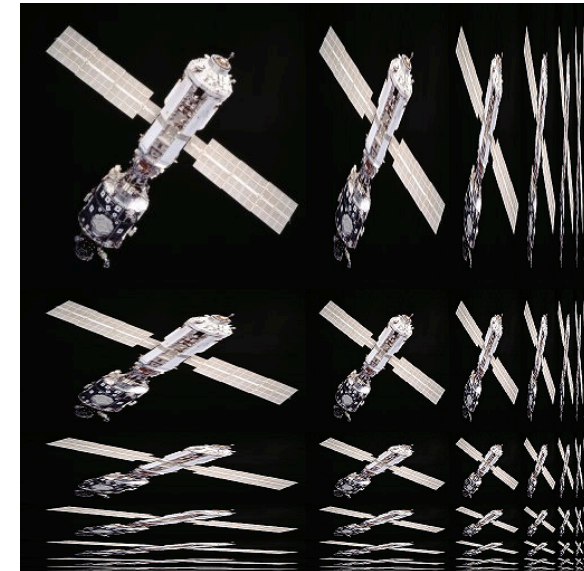
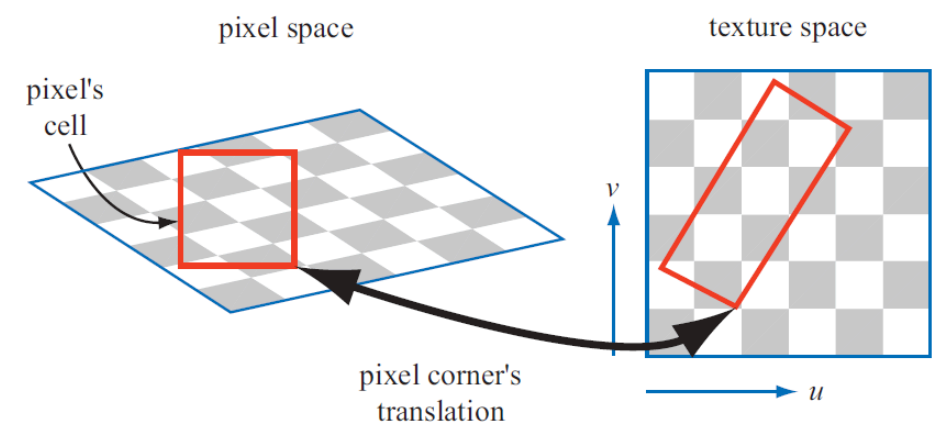
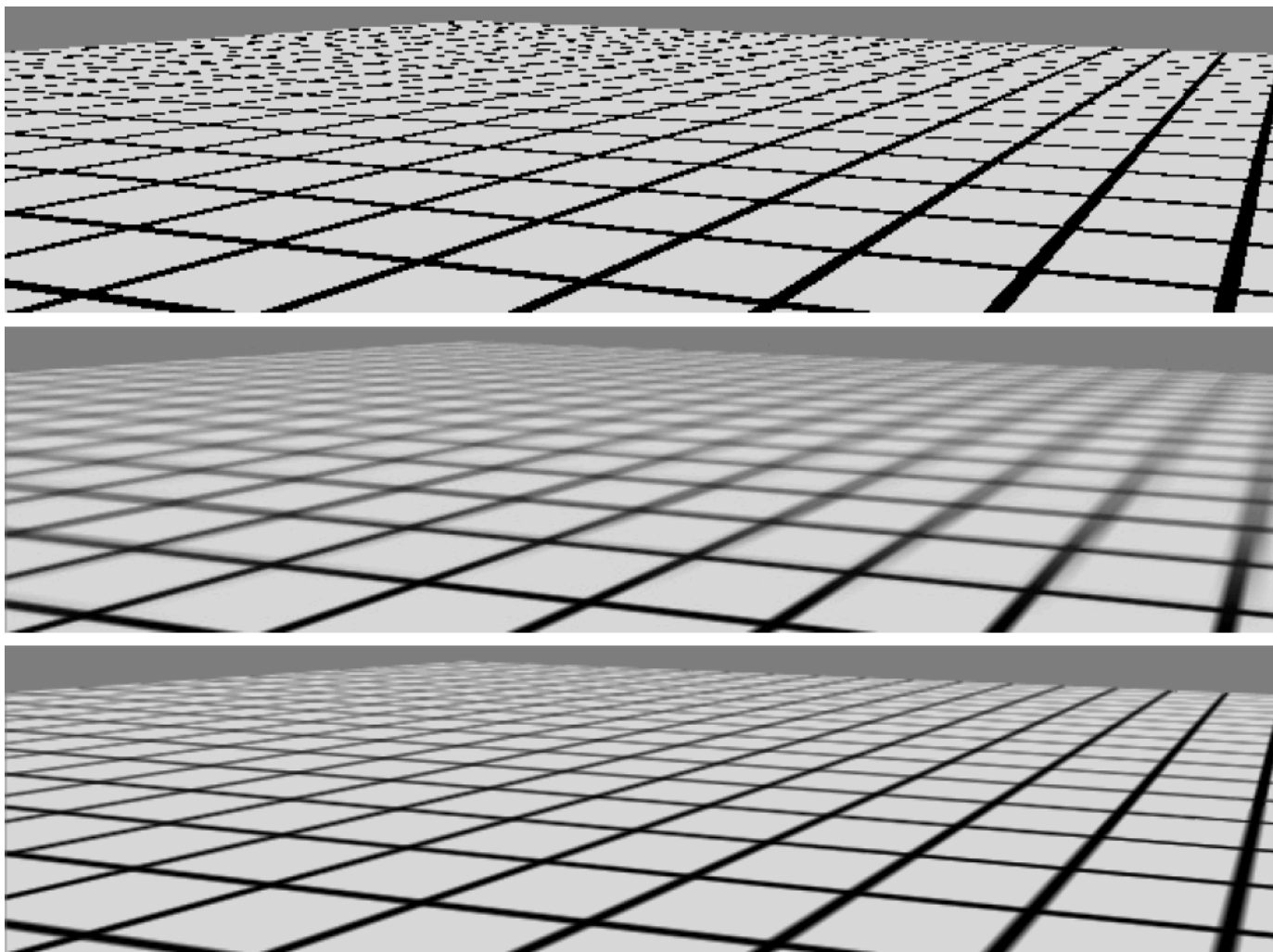
Este es el problema opuesto: cuando los téxeles son muy pequeños en relación a los píxeles. Esto significa que el pixel cubre varios téxeles. Si al momento de calcular el color de un fragmento tomamos el téxel que indican sus coordenadas st , el problema será que dos fragmentos consecutivos tomarán filas/columnas de téxeles que no lo son, descartando toda la información entre medio. Por ejemplo, si la textura tiene 100 columnas, y dos fragmentos consecutivos tienen coordenadas $s=0.1$ y $s=0.2$, un filtrado Nearest toma los colores de las columnas de téxeles 10 y 20, y en la imagen no se ve representada la información de las columnas 11 a 19. Si es linear, se consideran 4 columnas (se interpola entre dos para cada fragmento), pero aún sí se pierde información, ya que estos fragmentos "cubren" 20 columnas. Hacer que el shader calcule el color recorriendo todos los téxeles que corresponda sería muy caro.

Una técnica habitual para resolver esto es utilizar "MipMaps". Esto es, generar (al momento de cargar la textura) versiones escaladas de la imagen original, para tener distintas resoluciones disponibles y al momento de muestrear la textura para un cierto fragmento, elegir aquella versión donde los téxeles tengan un tamaño más parecido al de los píxeles.

Las versiones escaladas suelen generarse simplemente reduciendo recursivamente la resolución a la mitad. En cada nueva versión (nivel de MipMap) el téxel resultará en pantalla cuatro veces más grande (el doble en ancho y en alto) que el anterior. Si en un fragmento se toma por ejemplo un color del 3er nivel, dado que se generó como promedio de 4 téxeles del 2do nivel, y a su vez cada uno de ellos como promedio de 4 del primero, estamos indirectamente promediando información de 16 téxeles originales. Si utilizamos la bilineal en ese primer nivel, estaríamos considerando información de 64 téxeles originales.

Esta técnica obviamente requiere más memoria, aunque la diferencia no es tan grande como se podría pensar. La imagen de abajo a la derecha muestra que con un 50% más de memoria hay espacio de sobra para guardar todas las reducciones de tamaño hasta llegar a una imagen de un pixel.

REDUCCIÓN O MINIFICACIÓN: MIPMAPS



⚠ además del "tamaño" está el problema de la "alineación" del téxel

En la realidad el problema no es tan simple de resolver. Si bien la idea presentada mejora mucho el resultado, también hace que "desenfoque" la textura cuando los téxeles y píxeles además de no coincidir en tamaño, están desalineados, o la proyección hace que el téxel tenga una relación de aspecto diferente. Técnicas para resolver esto incluyen generar más versiones de la imagen con diferentes escalados en cada eje, y otras ideas más complejas que no tendremos tiempo de analizar. Se suele referir a estas mejoras como "filtrado anisotrópico" y suele estar disponible al programar como una opción más al configurar el sampler.

MODOS DE APLICACIÓN/MEZCLA

- ¿Cómo lo mezclamos con el color base o el material?
 - Modulate
 - Textura + Phong
 - Texturas + PBR



Ahora ya hemos definido cómo obtener las coordenadas de textura y cómo a partir de ellas obtener un "color". Resta hablar de cómo utilizar ese "color".

Lo más habitual será tratarlo como un verdadero color y combinarlo con los cálculos de iluminación. Si la textura dice que un fragmento debe verse verde, en realidad que efectivamente se vea de ese verde o más claro/oscuro dependerá de cómo le de la luz al objeto en ese punto. Pensando en el modelo de Phong, sería razonable utilizar el color que se obtiene de la textura en los parámetros K_a y K_d (las constantes del material para las componentes ambiente y difusa, que son las que le dan el color al objeto).

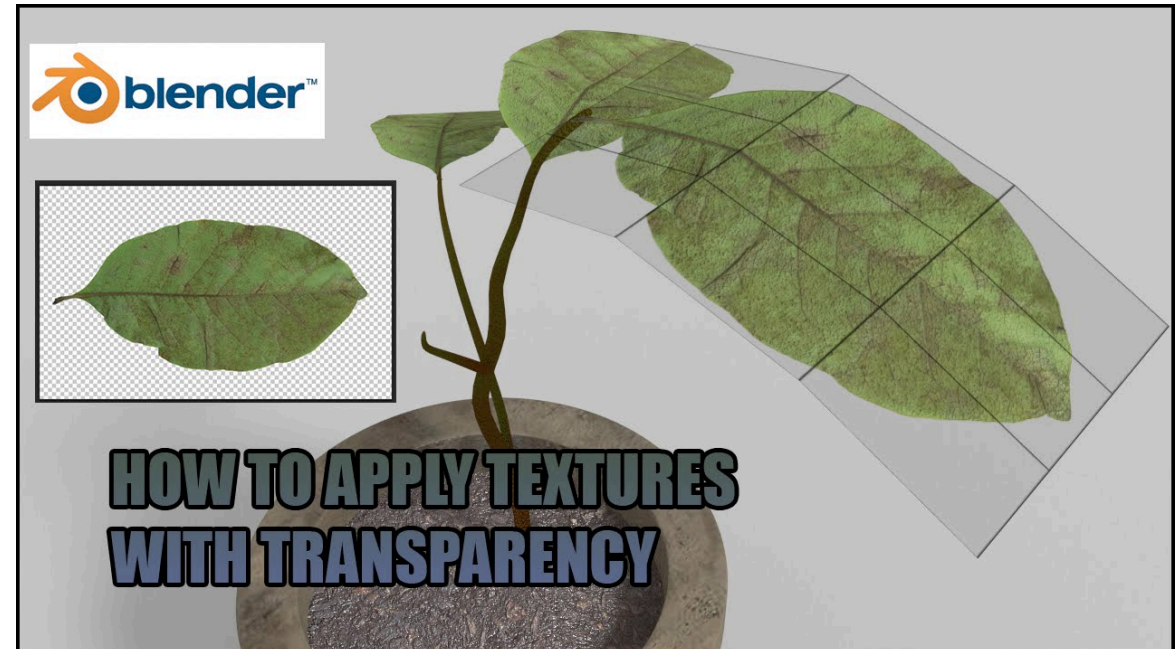
Pero se debe notar que la imagen de la textura se puede ver como una matriz donde cada "télcel" guarda 3 o 4 valores reales. Usualmente les asignamos el significado de color RGB, o RGBA, pero son en realidad valores que el shader puede interpretar como quiera. Se podría, por ejemplo, siguiendo con el modelo de Phong, usar una textura con canal alpha pero guardar en ese 4to valor por télcel algo que nos permita ajustar el brillo en lugar de la transparencia (un coeficiente con el cual ajustar q y K_s).

En un modelo de iluminación más complejo que Phong, donde se requieren otros parámetros para el material, se suelen utilizar dos o más texturas para codificarlos.

Entonces, el color de la textura puede usarse para algo tan simple como asignar color directamente al fragmento, o para codificar allí cualquier otra información que permita hacer trucos o cálculos más interesantes.

MODOS DE APLICACIÓN/MEZCLA

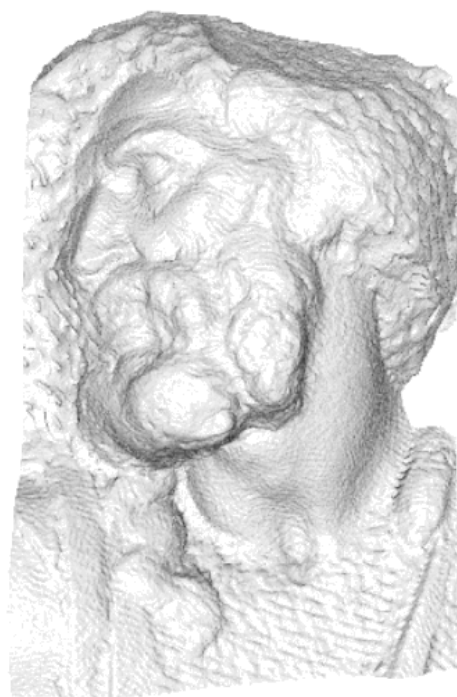
- ¿Para qué utilizamos el canal alpha?
- Textura transparente vs. Objeto transparente



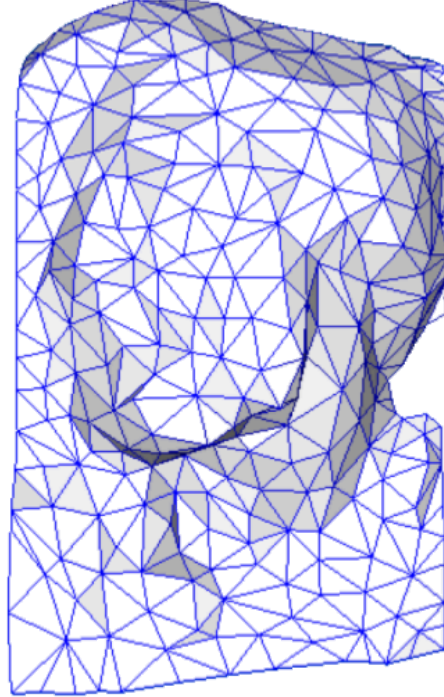
Supongamos ahora que tenemos una textura con canal alpha y que efectivamente usamos ese alpha para transparencia. No es lo mismo pensar que $\alpha=0$ hace a la textura transparente (no al objeto, sino como una calcomanía), que pensar que hace al objeto transparente. La segunda idea se utiliza mucho para cosas como vegetación: las hojas son casi planas y es mucho más fácil/barato darles "forma" con una textura de hoja con fondo transparente sobre un rectángulo (solo dos primitivas), que definiéndola geométricamente (muchos vértices y triángulos que copien el contorno).

MODOS DE APLICACIÓN/MEZCLA

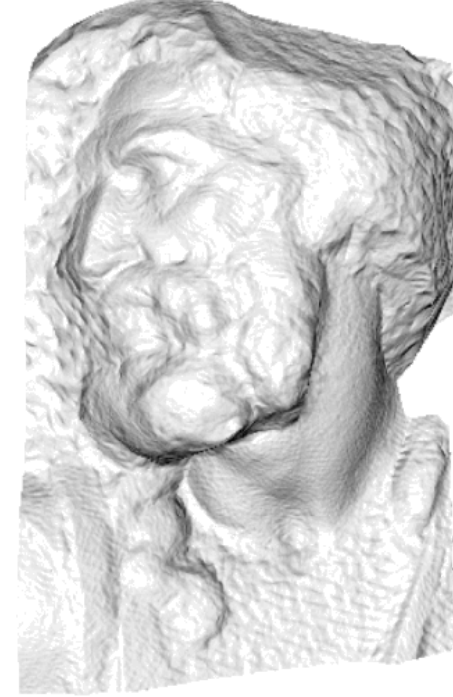
- ¿Otros usos?  *en lugar de "color" se puede guardar otra cosa*
- Normales
- Profundidad/desplazamiento



original mesh
4M triangles



simplified mesh
500 triangles

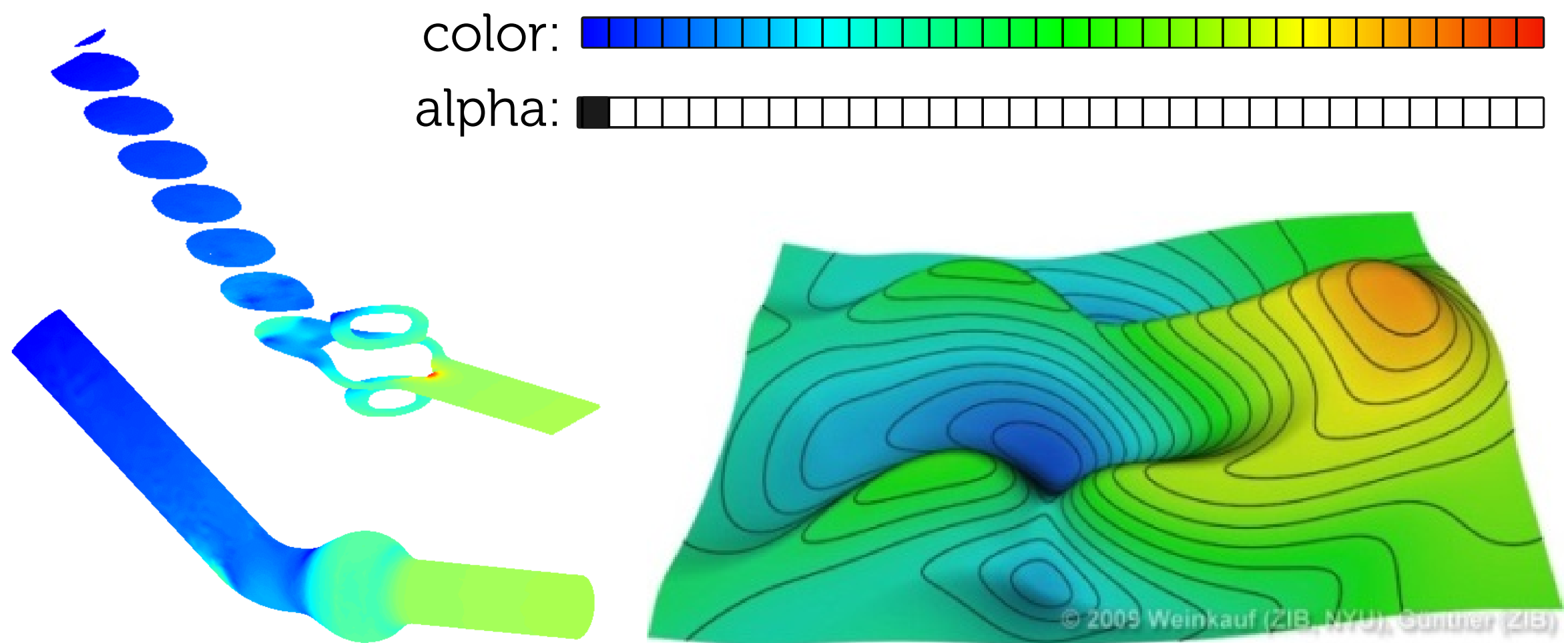


simplified mesh
and normal mapping
500 triangles

Entre los usos de textura para guardar otra cosa que no sea color, uno de los más habituales es el caso de "Bump mapping" (o "Normal mapping", son ideas similares). La idea es alterar la normal en base al "color" de la textura. Alterar la normal sería "casi" como variar la geometría: si en una geometría plana calculamos iluminación utilizando las normales de una geometría similar pero con cierto relieve, el resultado será parecido. En los canales RGB de una textura de normales, en lugar de color se codifican otros valores (como las componentes de la normal, o valores que indican cuánto perturbar a la normal original de la geometría en cada punto).

En la imagen de la izquierda hay una geometría con mucho detalle (y 4 millones de primitivas), mientras en la imagen de la derecha al detalle lo genera una textura (y solo usa 500 primitivas). El resultado visualmente no es tan diferente como sí lo es el costo computacional.

TEXTURAS 1D

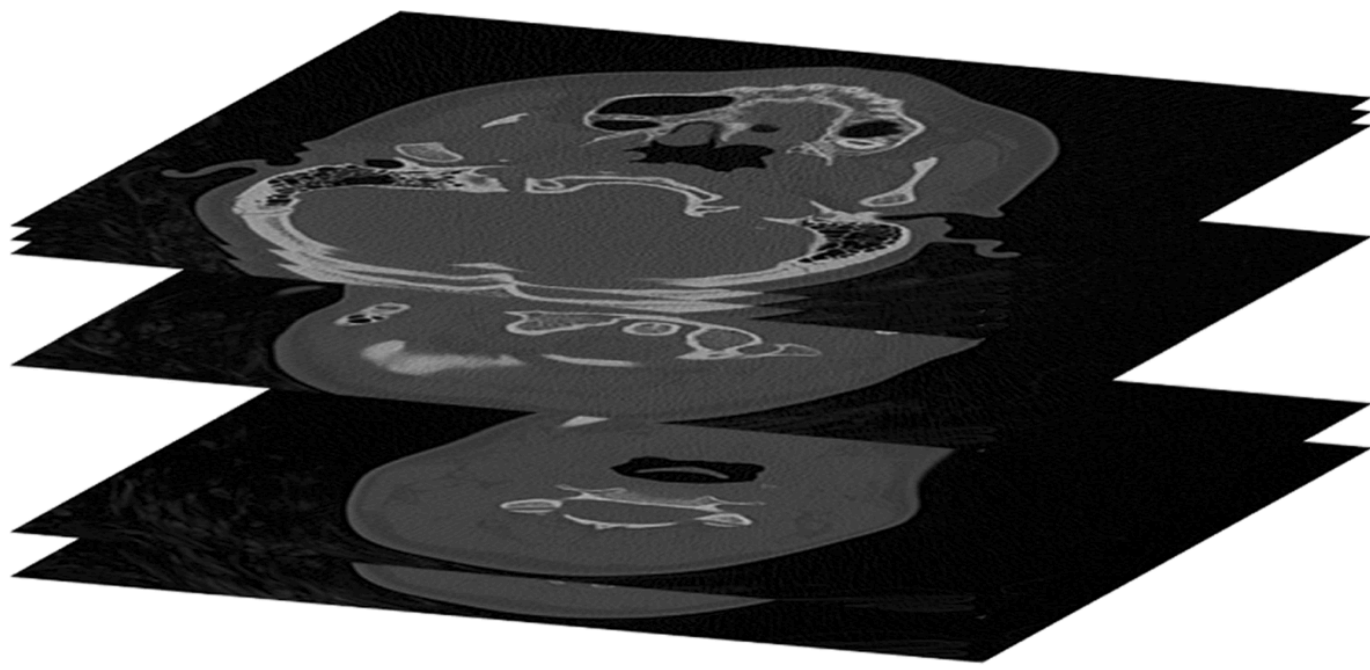


Hasta ahora hablamos siempre de las texturas como imágenes: matrices de texeles. Pero estas "matrices" podrían tener más o menos dimensiones (3D o 1D). Una textura 1D se puede pensar como una imagen de $1 \times N$, donde entonces no importa el mapeo de t , solo solo interesa mapear s .

Es habitual utilizar texturas 1D para visualización científica o similar. Si tenemos por ejemplo un valor de temperatura calculado en cada vértice y queremos graficar la temperatura en la superficie, utilizamos el valor de temperatura del vértice casi directamente como coordenada de textura (solo se ajusta la escala) sobre una textura con la escala de colores deseada (0=frio, 1=caliente) y dejamos que al renderizar la interpolación en los fragmentos interpole esa "temperatura".

Si usamos una textura con un solo primer texel opaco y el resto transparente, al aplicarla con el mismo mecanismo se generará una isolínea donde la temperatura sea 0. Reajustando la escala y utilizando Repeat podemos visualizar múltiples isolíneas sin realmente tener que calcularlas (es la GPU la que lo hace por nosotros).

TEXTURAS 3D



✅ la "imagen" puede ser una función $f(s, t, u)$ en lugar de una matriz

También existe el concepto de textura 3D, donde tenemos "vóxeles" en lugar de téxeles, ya que cada color en esa matriz de 3 dimensiones representa ahora un pequeño volumen y no un área.

Puede pensarse como que es una "pila" de imágenes, como en el caso de muchos estudios médicos 3D, donde lo que se hace en realidad es tomar "fotos 2D" a diferentes profundidades (para cada z). Sin embargo, tener esa información efectivamente como una matriz implica un gran volumen de memoria.

En muchos casos esa información es generada por ecuaciones (lo que se conoce como "texturas procedurales", como el mármol de la tetera) y no almacenada como imagen raster 3D (vóxeles).

¿QUÉ HAY QUE SABER?

- ▶ Concepto de textura y **mapeo**
 - ▶ Mapeo manual vs automático
 - ▶ Mapeos UV, plano, y en 2 partes (cilíndrico, esférico, cúbico)
 - ▶ Usos de texturas 1D
- ▶ Aplicación:
 - ▶ Corrección de la coordenadas: clamp vs. repeat
 - ▶ Modos de aplicación (ej: combinar con Phong).
 - ▶ Otras: normal/bump mapping (solo idea, sin detalles)
- ▶ Filtrado: El problema de la minificación/magnificación
 - ▶ Nearest/Linear
 - ▶ Mipmaps